

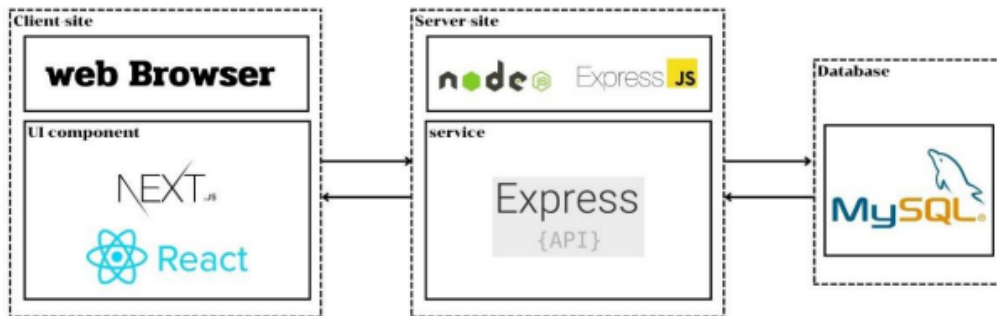
คู่มือสำหรับนักพัฒนา

เว็บไซต์สื่อการสอนภาษาแอสคิวแอล

SQL-Tutor

คู่มือสำหรับพัฒนา

โครงสร้างของระบบ SQL-Tutor System



รูปโครงสร้างของระบบ

เทคโนโลยีหลักที่ใช้ในโปรเจค:

โปรเจกต์นี้ถูกพัฒนาโดยใช้เทคโนโลยีหลัก 3 ส่วน ดังนี้:

1. ส่วนแสดงผลฝั่งผู้ใช้งาน (Frontend - Web):

- เฟรมเวิร์ก: Next.js (A React Framework) - ใช้สำหรับพัฒนาส่วนติดต่อผู้ใช้งานบนเว็บไซต์ โดยอาศัย React library เป็นพื้นฐาน

2. ส่วนประมวลผลฝั่งเซิร์ฟเวอร์ (Back-end):

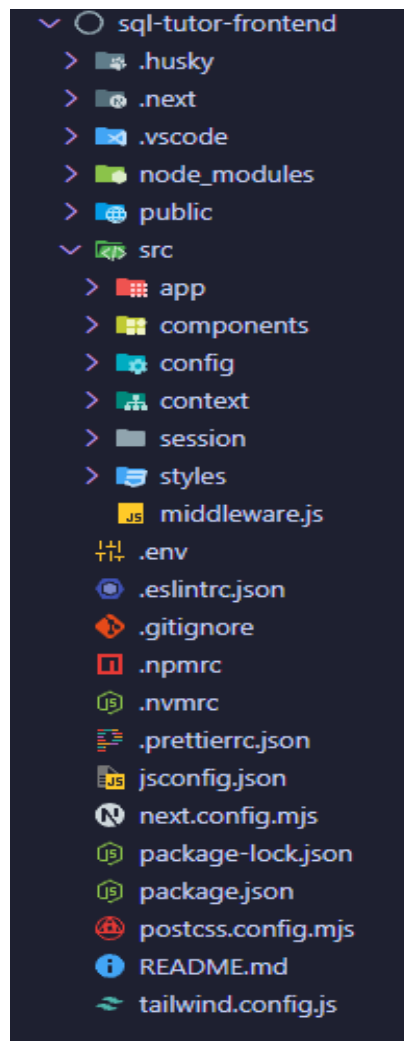
- เฟรมเวิร์ก: Express.js (A Node.js Framework) - ใช้สำหรับสร้าง API และจัดการตรรกะทางของแอปพลิเคชันบนสภาพแวดล้อม Node.js

3. ฐานข้อมูล (Database):

- ระบบจัดการฐานข้อมูล: MySQL - ใช้สำหรับจัดเก็บและจัดการข้อมูล

ส่วนของ Web Application

ภาพรวมโครงสร้างของ Web Application



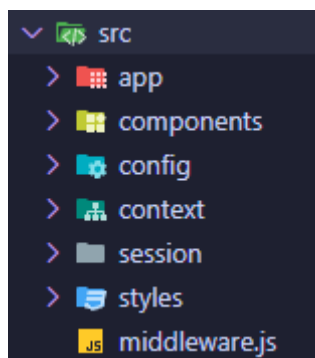
โครงสร้างของ Frontend – Web Application

- .husky ใช้สำหรับ Git Hooks เพื่อรันสคริปต์ก่อน commit หรือ gpush โค้ด
- .next โฟลเดอร์ที่ Next.js สร้างขึ้นเพื่อเก็บไฟล์ build, ไฟล์แคช, และข้อมูลอื่นๆ ที่เกี่ยวข้องกับการทำงานของแอปพลิเคชัน

- .vscode โฟลเดอร์ที่เก็บการตั้งค่าเฉพาะสำหรับโปรแกรมแก้ไขโค้ด Visual Studio Code
- node_modules โฟลเดอร์ที่เก็บ Dependencies หรือ Library ต่างๆ ที่โปรเจกใช้งาน ซึ่งติดตั้งผ่าน npm หรือ yarn
- public โฟลเดอร์สำหรับเก็บไฟล์ Static Assets เช่น รูปภาพ, ฟอนต์, หรือไฟล์อื่นๆ ที่ต้องการให้เข้าถึงได้โดยตรงผ่าน URL
- src โฟลเดอร์หลักของ Source Code ในโปรเจก Next.js
- .env ไฟล์สำหรับเก็บ Environment Variables ที่ใช้ในการพัฒนาและ Production (ควรถูกเพิ่มใน .gitignore เพื่อป้องกันการ Commit ข้อมูลลับ)
- .eslintrc.json ไฟล์กำหนดค่าสำหรับ ESLint ซึ่งเป็นเครื่องมือตรวจสอบ Code Style และหา Error ใน JavaScript/JSX
- .gitignore ไฟล์ที่ระบุไฟล์และโฟลเดอร์ที่ไม่ต้องการให้ Git ติดตามและ Commit
- .npmrc ไฟล์กำหนดค่าสำหรับ npm (Node Package Manager)
- .nvmrc ไฟล์ที่ระบุ Node.js version ที่แนะนำให้ใช้ในโปรเจก (ใช้ร่วมกับ Node Version Manager)
- .prettierrc.json ไฟล์กำหนดค่าสำหรับ Prettier ซึ่งเป็นเครื่องมือจัดรูปแบบโค้ดอัตโนมัติ
- jsconfig.json หรือ tsconfig.json ไฟล์กำหนดค่าสำหรับ JavaScript หรือ TypeScript (ในกรณีนี้อาจเป็น JavaScript) ใช้เพื่อกำหนดค่าภาษาและตัวเลือกต่างๆ สำหรับ Editor และเครื่องมืออื่นๆ
- next.config.mjs ไฟล์กำหนดค่าหลักสำหรับ Next.js เช่น การตั้งค่า Build, Webpack, Environment Variables, และอื่นๆ
- package-lock.json (หรือ yarn.lock) ไฟล์ที่บันทึกเวอร์ชันที่แน่นอนของ Dependencies ที่ติดตั้ง เพื่อให้การติดตั้งในสภาพแวดล้อมอื่นเป็นไปในรูปแบบเดียวกัน
- package.json ไฟล์ที่เก็บข้อมูล metadata ของโปรเจก เช่น ชื่อ, เวอร์ชัน, scripts (คำสั่งต่างๆ ที่ใช้รันโปรเจก), และ Dependencies

- postcss.config.mjs ไฟล์กำหนดค่าสำหรับ PostCSS ซึ่งเป็นเครื่องมือสำหรับแปลง CSS ด้วย Plugins ต่างๆ (มักใช้ร่วมกับ Tailwind CSS)
- README.md ไฟล์ Markdown ที่อธิบายเกี่ยวกับโปรเจกต์ วิธีการติดตั้งและใช้งาน
- tailwind.config.js ไฟล์กำหนดค่าสำหรับ Tailwind CSS ซึ่งเป็น CSS Utility-First Framework

โฟลเดอร์ src

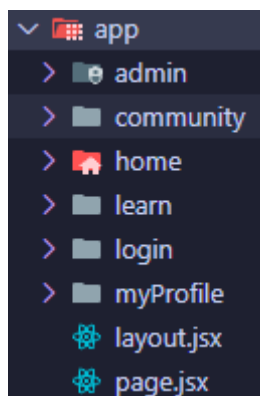


โครงสร้างของโฟลเดอร์ src

- /app โฟลเดอร์นี้เป็นหัวใจสำคัญของ Next.js App Router (ตั้งแต่ Next.js 13) ใช้สำหรับการจัดการ:
 - Routing กำหนดเส้นทาง (Routes) ต่างๆ ของแอปพลิเคชันโดยใช้โครงสร้างโฟลเดอร์
 - Layouts กำหนด Layout ส่วนกลางและ Layout เฉพาะสำหรับบางส่วนของแอปพลิเคชัน
 - Pages สร้างเพจ (Pages) ต่างๆ ของแอปพลิเคชัน
 - API Routes สร้าง API endpoints
- /components โฟลเดอร์นี้ใช้สำหรับเก็บ React Components ที่ถูกนำมาใช้ซ้ำในส่วนต่างๆ ของแอปพลิเคชัน เพื่อความเป็นระเบียบและง่ายต่อการจัดการ
- /config โฟลเดอร์นี้มีไว้สำหรับเก็บไฟล์ Configuration ต่างๆ ที่ใช้ในการปรับแต่งการทำงานของแอปพลิเคชัน เช่น การตั้งค่า API keys, environment variables, หรือ configurations อื่นๆ
- /context โฟลเดอร์นี้ใช้สำหรับการจัดการ Context API ใน React ซึ่งเป็นวิธีที่มีประสิทธิภาพในการจัดการ State ที่จำเป็นต้องใช้ร่วมกันในหลายๆ Components โดยไม่ต้องส่ง Props ผ่านหลายๆ ระดับ

- /session โฟลเดอร์นี้เกี่ยวข้องกับการจัดการ Session หรือระบบ Authentication ของผู้ใช้งาน เช่น การเก็บข้อมูลผู้ใช้ที่ Login, การจัดการ Cookies หรือ Tokens ที่เกี่ยวข้องกับการ Login
- /styles โฟลเดอร์นี้ใช้สำหรับเก็บ Stylesheets ที่ใช้ในการตกแต่งหน้าตาของแอปพลิเคชัน ซึ่งอาจเป็น CSS Modules (.module.css), Global CSS (.css), หรือ CSS-in-JS libraries
- /middleware.js ไฟล์นี้ใช้สำหรับกำหนด Middleware Functions ซึ่งจะทำงานก่อนที่ Request จะเข้าถึง Routes ต่างๆ ภายในโฟลเดอร์ app ทำให้สามารถตรวจสอบสิทธิ์, Redirect ผู้ใช้, หรือทำ Process อื่นๆ ก่อนการเข้าถึงหน้าเพจได้

โฟลเดอร์ app

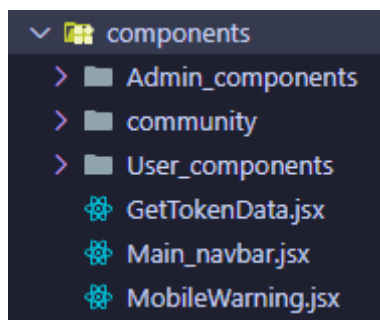


โครงสร้างของโฟลเดอร์ app

- /admin โฟลเดอร์นี้ใช้เก็บในส่วนของหน้า Admin ของแอปพลิเคชัน โดยภายในจะมี Layout และ Pages ต่างๆ
- /community โฟลเดอร์นี้ใช้เก็บในส่วน Community ของแอปพลิเคชัน เช่น กระดาน, ฟอรัม, หรือพื้นที่สำหรับผู้ใช้งานร่วมกัน
- /home โฟลเดอร์นี้ใช้เก็บสำหรับหน้า Home หรือหน้าแรกของแอปพลิเคชัน
- /learn โฟลเดอร์นี้ใช้เก็บส่วนของการเรียนรู้ ของแอปพลิเคชัน
- /login โฟลเดอร์นี้ใช้เก็บสำหรับหน้า Login หรือหน้าสำหรับเข้าสู่ระบบของแอปพลิเคชัน

- /myProfile โฟลเดอร์นี้ใช้เก็บสำหรับหน้า Profile ส่วนตัวของผู้ใช้งาน
- layout.jsx ไฟล์นี้คือ Layout Component ที่ใช้สำหรับกำหนดโครงสร้าง Layout ส่วนกลางของทุก Pages ภายในโฟลเดอร์ app หรือ Layout เฉพาะสำหรับบางส่วนของแอปพลิเคชัน Layout จะห่อหุ้ม (wrap) Pages ต่างๆ และคงส่วนประกอบที่ไม่เปลี่ยนแปลง เช่น Navigation Bar หรือ Footer ไว้
- page.jsx ไฟล์นี้คือ Page Component ที่ใช้สำหรับแสดงเนื้อหาหลักของแต่ละ Route ภายในโฟลเดอร์ app โดย page.jsx ที่อยู่ใน Root ของ app จะเป็น Page หลักของเว็บไซต์

โฟลเดอร์ components



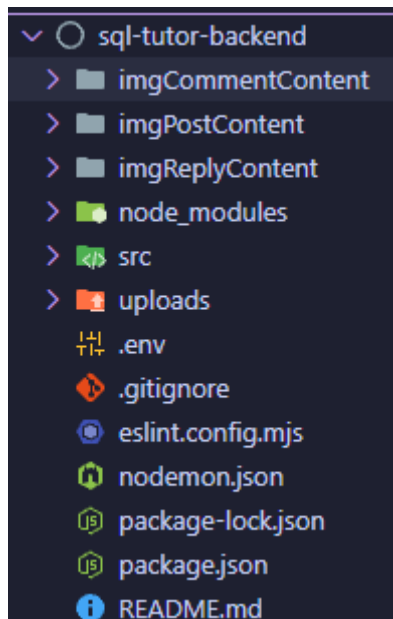
โครงสร้างของ components

- /Admin_components โฟลเดอร์ย่อยภายใน components จะเก็บ Components ที่ใช้เฉพาะในส่วน Admin ของแอปพลิเคชัน เพื่อแยกส่วนการทำงานและ Components ที่เกี่ยวข้องกับผู้ดูแล
- /community โฟลเดอร์ย่อยภายใน components จะเก็บ Components ที่เกี่ยวข้องกับส่วน Community หรือพื้นที่สำหรับผู้ใช้งานร่วมกันของแอปพลิเคชัน

- /User_components โฟลเดอร์ย่อยภายใน components จะเก็บ Components ที่เกี่ยวข้องกับการทำงานหรือแสดงผลในส่วนของผู้ใช้งานทั่วไป
- /GetTokenData.jsx ไฟล์นี้เป็น React Component มีหน้าที่ในการดึงข้อมูล Token และนำข้อมูลนั้นมาใช้ในการตรวจสอบสิทธิ์หรือดึงข้อมูลผู้ใช้งาน
- /Main_navbar.jsx ไฟล์นี้เป็น React Component เป็นส่วนประกอบของ Navigation Bar หลักของแอปพลิเคชันและแสดงเมนูต่างๆ สำหรับผู้ใช้งาน
- /MobileWarning.jsx ไฟล์นี้เป็น React Component ใช้ในการแสดงคำเตือนหรือข้อความแจ้งเตือนสำหรับผู้ใช้งานที่เข้าชมเว็บไซต์บนอุปกรณ์มือถือ จะแจ้งว่าเว็บไซต์ยังไม่รองรับการใช้งานบนมือถือหรือแนะนำให้ใช้งานบนเดสก์ท็อป

ส่วนของ API Server

ภาพรวมของ API Server

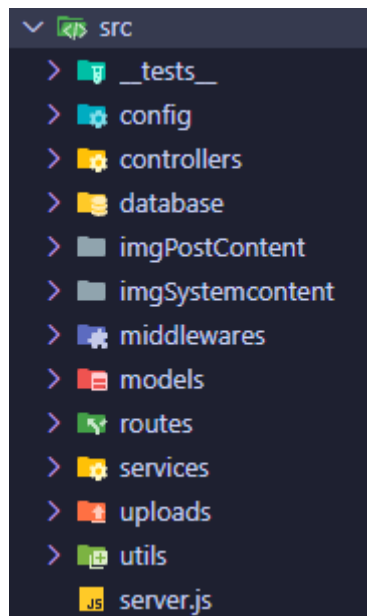


ภาพรวมของ API Server

- /imgCommentContent โฟลเดอร์นี้ใช้สำหรับเก็บรูปภาพหรือเนื้อหาที่เป็นรูปภาพที่เกี่ยวข้องกับความคิดเห็น (comments) ในระบบ
- /imgPostContent โฟลเดอร์นี้ใช้สำหรับเก็บรูปภาพหรือเนื้อหาที่เป็นรูปภาพที่เกี่ยวข้องกับโพสต์ (posts) ในระบบ
- /imgReplyContent โฟลเดอร์นี้ใช้สำหรับเก็บรูปภาพหรือเนื้อหาที่เป็นรูปภาพที่เกี่ยวข้องกับการตอบกลับ (replies) ในระบบ
- /node_modules โฟลเดอร์นี้เป็นมาตรฐานสำหรับโปรเจกต์ Node.js ใช้สำหรับเก็บ Dependencies หรือ Packages ที่ถูกติดตั้งผ่าน npm หรือ yarn
- /src โฟลเดอร์หลักของ Source Code
- /uploads โฟลเดอร์นี้ใช้สำหรับเก็บไฟล์ต่างๆ ที่ผู้ใช้งานอัปโหลดเข้ามาในระบบ
- .env ไฟล์นี้ใช้ในการจัดการ Environment Variables หรือตัวแปรสภาพแวดล้อม
- .gitignore ไฟล์นี้ใช้สำหรับระบุไฟล์และโฟลเดอร์ที่ไม่ต้องการให้ Git ติดตาม (track) และรวมอยู่ในการ Repository
- eslint.config.mjs ไฟล์ Configuration สำหรับ ESLint ซึ่งเป็นเครื่องมือสำหรับตรวจสอบรูปแบบโค้ด JavaScript เพื่อให้โค้ดมีคุณภาพและสไตล์ที่สม่ำเสมอ (.mjs บ่งชี้ว่าเป็น Module JavaScript)

- nodemon.json ไฟล์ Configuration สำหรับ Nodemon ซึ่งเป็นเครื่องมือที่ช่วยในการพัฒนา Node.js โดยจะ Restart เซิร์ฟเวอร์โดยอัตโนมัติเมื่อมีการเปลี่ยนแปลงไฟล์
- package-lock.json ไฟล์นี้จะบันทึกเวอร์ชันที่แน่นอนของ Dependencies ที่ถูกติดตั้งในโปรเจกต์ เพื่อให้มั่นใจว่าทุกคนที่ทำงานในโปรเจกต์จะใช้ Dependencies เวอร์ชันเดียวกัน
- package.json ไฟล์นี้เป็นของโปรเจกต์ Node.js ใช้สำหรับเก็บข้อมูล Metadata ของโปรเจกต์ เช่น ชื่อ, เวอร์ชัน, Scripts (คำสั่งต่างๆ ที่ใช้รันโปรเจกต์), และ Dependencies ที่โปรเจกต์ใช้งาน
- README.md ไฟล์นี้ใช้สำหรับอธิบายรายละเอียดของโปรเจกต์ วิธีการติดตั้ง และวิธีการใช้งาน

โฟลเดอร์ src



โครงสร้างของ components

- config โฟลเดอร์สำหรับเก็บไฟล์ Configuration ต่างๆ ของแอปพลิเคชัน เช่น การตั้งค่าฐานข้อมูล, การตั้งค่า API keys, หรือการตั้งค่าอื่นๆ
- controllers โฟลเดอร์สำหรับเก็บ Controllers ซึ่งมีหน้าที่ในการจัดการ Logic ของ Request ที่เข้ามาไปยัง Routes ต่างๆ และส่งต่อการทำงานไปยัง Services หรือ Models
- database โฟลเดอร์เก็บไฟล์กับการจัดการฐานข้อมูล กับไฟล์ Configuration การเชื่อมต่อ

- middlewares โฟลเดอร์สำหรับเก็บ Middleware Functions ซึ่งจะทำงานก่อนหรือหลังการเข้าถึง Routes ต่างๆ ใช้สำหรับตรวจสอบสิทธิ์, Logging, Data Validation
- models โฟลเดอร์สำหรับเก็บ Models ซึ่งเป็นตัวแทนของข้อมูลในฐานข้อมูล และมีหน้าที่ในการจัดการการเข้าถึงและจัดการข้อมูล
- routes โฟลเดอร์สำหรับเก็บไฟล์ที่กำหนด Routes ต่างๆ ของ API โดยจะจับคู่ URL กับ Controller ที่เหมาะสม
- services โฟลเดอร์สำหรับเก็บ Services ซึ่งมีหน้าที่ในการจัดการ Business Logic ของแอปพลิเคชัน โดยมักจะถูกเรียกใช้จาก Controllers
- utils โฟลเดอร์สำหรับเก็บ Utility Functions หรือ Helper Functions ที่ใช้ซ้ำในส่วนต่างๆ ของแอปพลิเคชันไฟล์
- server.js ไฟล์นี้เป็นจุดเริ่มต้น (Entry Point) ของแอปพลิเคชัน Node.js ซึ่งจะทำการตั้งค่า Server, Load Modules ต่างๆ, และเริ่มต้นการทำงานของแอปพลิเคชัน

API Documentation

ไฟล์ AuthRoutes.js

Method	Endpoint	Description
POST	/login	สำหรับเข้าสู่ระบบ
POST	/register	สำหรับลงทะเบียน
GET	/user/:accountId	ดูข้อมูลผู้ใช้รายบุคคล
POST	/forget-password	ดูข้อมูลผู้ใช้ทั้งหมด
PUT	/change-password	สำหรับลืมหรีตผ่าน
PUT	/update-profile	เปลี่ยนรหัสผ่าน
GET	/users	อัปเดตข้อมูลโปรไฟล์
DELETE	/user/:accountId	ลบบัญชีผู้ใช้

ไฟล์ communityRoutes.js

Method	Endpoint	Description
--------	----------	-------------

POST	/create	สร้างโพสต์ใหม่
GET	/all	ดูโพสต์ทั้งหมด
GET	/post/:id	ดูโพสต์เดียวๆ
PUT	/post/:id	แก้ไขโพสต์
DELETE	/post/:id	ลบโพสต์
POST	/comment	สร้างคอมเมนต์
PUT	/comment/:commentId/helpful	มาร์คคอมเมนต์ว่ามีประโยชน์
PUT	/comment/:commentId	แก้ไขคอมเมนต์
DELETE	/comment/:commentId	ลบคอมเมนต์
POST	/reply	สร้างการตอบกลับ
GET	/reply/:commentId	ดูการตอบกลับของคอมเมนต์
GET	/reply/nested/:parentReplyId	ดูการตอบกลับซ้อนการตอบกลับ
PUT	/reply/:replyId	แก้ไขการตอบกลับ
DELETE	/reply/:replyId	ลบการตอบกลับ

ไฟล์ `courseRoute.js`

Method	Endpoint	Description
GET	/topics	ดึงข้อมูลหัวข้อหลักทั้งหมด
GET	/topic/:id	ดึงข้อมูลหัวข้อหลักตาม ID
GET	/subtopics/:id	ดึงข้อมูลหัวข้อย่อยตาม ID
GET	/contents/:id	ดึงข้อมูลเนื้อหาตาม ID
POST	/topics	สร้างหัวข้อหลักใหม่
PUT	/topics/:id	อัปเดตหัวข้อหลัก
POST	/topics/:M_topic_id/subtopics	สร้างหัวข้อย่อยใหม่ในหัวข้อหลัก
PUT	/subtopics/:id	อัปเดตหัวข้อย่อย
PUT	/contents/:id	อัปเดตเนื้อหา
POST	/contents/:contentId/images	อัปโหลดรูปภาพในเนื้อหา
DELETE	/topics/:id	ลบหัวข้อหลัก
DELETE	/subtopics/:id	ลบหัวข้อย่อย
POST	/execute	รันโค้ด SQL
POST	/answers	บันทึกคำตอบที่ถูกต้อง

POST	/contents/:contentId/check	ตรวจสอบคำตอบของผู้ใช้
GET	/topics-with-progress	ดึงข้อมูลความก้าวหน้าของผู้ใช้ในทุกหัวข้อ
PUT	/progress/:contentId	อัปเดตความก้าวหน้าของผู้ใช้
GET	/topics-progress	ดึงข้อมูลภาพรวมความก้าวหน้าทั้งหมด
POST	/maintopic/reorder	จัดลำดับหัวข้อหลักใหม่
POST	/subtopic/reorder	จัดลำดับหัวข้อย่อยใหม่
POST	/user/reset-db	รีเซ็ตฐานข้อมูลของผู้ใช้